11 Distributed tracing with Spring Cloud Sleuth and Zipkin

This chapter covers

- Using Spring Cloud Sleuth to inject tracing information into service calls
- Using log aggregation to see logs for distributed transactions
- Transforming, searching, analyzing, and visualizing log data in real time
- Understanding a user transaction as it flows across multiple service classes
- Customizing tracing information with Spring Cloud Sleuth and Zipkin

The microservices architecture is a powerful design paradigm for breaking down complex monolithic software systems into smaller, more manageable pieces. These pieces can be built and deployed independently of each other; however, this flexibility comes at a price—complexity.

Because microservices are distributed by nature, trying to debug where a problem occurs can be maddening. The distributed nature of the services means that we need to trace one or more transactions across multiple services, physical machines, and different data stores, and then try to piece together what exactly is going on. This chapter lays out several techniques and technologies for using distributed debugging. In this chapter, we look at the following:

- Using correlation IDs to link together transactions across multiple services
- Aggregating log data from various services into a single searchable source
- Visualizing the flow of a user transaction across multiple services to understand each part of the transaction's performance characteristics
- Analyzing, searching, and visualizing log data in real time using the ELK stack

To accomplish this, we will use the following technologies:

- *Spring Cloud Sleuth* (**https://cloud.spring.io/spring-cloud-sleuth/reference/html/**)—The Spring Cloud Sleuth project instruments our incoming HTTP requests with trace IDs (aka correlation IDs). It does this by adding filters and interacting with other Spring components to let the generated correlation IDs pass through to all the system calls.
- *Zipkin* (**https://zipkin.io/**)—Zipkin is an open source data-visualization tool that shows the flow of a transaction across multiple services. Zipkin allows us to break a transaction down into its component pieces and visually identify where there might be performance hotspots.
- *ELK stack* (**https://www.elastic.co/what-is/elk-stack**)—The ELK stack combines three open source tools—Elasticsearch, Logstash, and Kibana—that allow us to analyze, search, and visualize logs in real time.
  - Elasticsearch is a distributed analytics engine for all types of data (structured and non-structured, numeric, text-based, and so on).
  - Logstash is a server-side data processing pipeline that allows us to add and ingest data from multiple sources simultaneously and transform it before it is indexed in Elasticsearch.
  - Kibana is the visualization and data management tool for Elasticsearch. It provides charts, maps, and real-time histograms.

To begin this chapter, we'll start with the simplest of tracing tools: the correlation ID. We'll cover this in the following section.

NOTE Parts of this chapter rely on material covered in chapter 8 (mainly the material on the Spring Gateway response, pre-, and post-filters). If you haven't read chapter 8 yet, we recommend that you do so before you read this chapter.

11.1 Spring Cloud Sleuth and the correlation ID

We first introduced the concept of correlation IDs in chapters 7 and 8. A *correlation ID* is a randomly generated unique number or string assigned to a transaction as the transaction is initiated. As the transaction flows across multiple services, the correlation ID is propagated from one service call to another.

In the context of chapter 8, we used a Spring Cloud Gateway filter to inspect all incoming HTTP requests and inject a correlation ID into the request if one wasn't present. Once the correlation ID was present, we used a custom Spring HTTP filter on each one of our services to map the incoming variable to a custom `UserContext` object. With this object in place, we manually added the correlation ID to our log statements by appending the correlation ID to a log statement or, with a little work, adding the correlation ID directly

to Spring's Mapped Diagnostic Context (MDC). MDC is a map that stores a set of key-value pairs provided by the application that's inserted in the log messages.

In that chapter, we also wrote a Spring interceptor to ensure that all HTTP calls from a service would propagate the correlation ID by adding the correlation ID to the HTTP headers of outbound calls. Fortunately, Spring Cloud Sleuth manages all this code infrastructure and complexity for us. Let's go ahead and add Spring Cloud Sleuth to our licensing and organization services. We'll see that by adding Spring Cloud Sleuth to our microservices, we can

- Transparently create and inject a correlation ID into our service calls if one doesn't exist
- Manage the propagation of correlation IDs to outbound service calls so that the correlation ID for a transaction is automatically added
- Add the correlation information to Spring's MDC logging so that the generated correlation ID is automatically logged by Spring Boot's default SL4J and Logback implementation
- Optionally, publish the tracing information in the service call to the Zipkin distributed tracing platform

NOTE With Spring Cloud Sleuth, if we use Spring Boot's logging implementation, we'll automatically get correlation IDs added to the log statements we put in our microservices.

.1.1 Adding Spring Cloud Sleuth to licensing and organization

To start using Spring Cloud Sleuth in our two services (licensing and organization), we need to add a single Maven dependency to the pom.xml files in both services. The following shows you how to do this:

```
<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-starter-sleuth</artifactId>
</dependency>
```

These dependencies pull in all the core libraries needed for Spring Cloud Sleuth. That's it. Once they are pulled in, our service will now

- Inspect every incoming HTTP service and determine whether Spring Cloud Sleuth tracing information exists in the incoming call. If the Spring Cloud Sleuth tracing data is present, the tracing information passed into our microservice will be captured and made available to our service for logging and processing.
- Add Spring Cloud Sleuth tracing information to the Spring MDC so that every log statement created by our microservice will be added to the log.
- Inject Spring Cloud tracing information into every outbound HTTP call and Spring messaging channel message our service makes.

.1.2 Anatomy of a Spring Cloud Sleuth trace

If everything is set up correctly, any log statements written in our service application code will now include Spring Cloud Sleuth trace information. For example, figure 11.1 shows what the service's output would look like if we were to issue an HTTP GET on the following endpoint in the organization service:

```
http://localhost:8072/organization/v1/organization/95c0dab4-0a7e-48f8-805a-
➥ 0ba31c3687b8
```

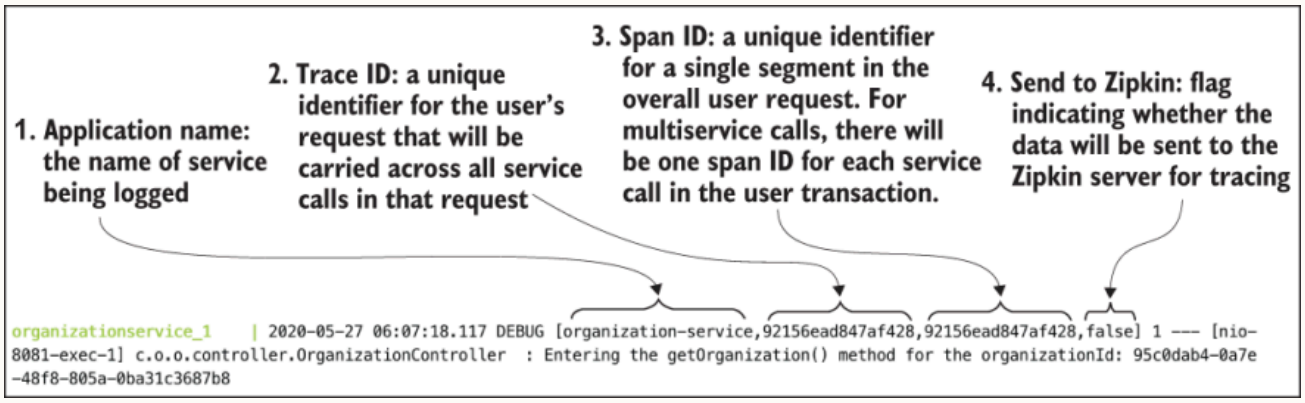


Figure 11.1 Spring Cloud Sleuth adds tracing information to each log entry written by our organization service. This data helps tie together service calls for a user's request.

Spring Cloud Sleuth adds four pieces of information to each log entry. These four pieces (numbered to correspond with the numbers in figure 11.1) are as follows:

1. *The application name of the service where the log entry is entered.* By default, Spring Cloud Sleuth uses the application name (`spring.application.name`) as the name that gets written in the trace.
2. *The trace ID, which is the equivalent term for correlation ID.* This is a unique number that represents an entire transaction.

3. *The span ID, which is a unique ID that represents part of the overall transaction.* Each service participating within the transaction will have its own span ID. Span IDs are particularly relevant if you integrate with Zipkin to visualize your transactions.
4. *Export, a true/false indicator that determines whether trace data is sent to Zipkin.* In high-volume services, the amount of trace data generated can be overwhelming and not add a significant amount of value. Spring Cloud Sleuth lets us determine when and how to send a transaction to Zipkin.

NOTE By default, any application flow starts with the same trace and span IDs.

Up to now, we've only looked at the logging data produced by a single service call. Let's look at what happens when you also make a call to the licensing service. Figure 11.2 shows the logging output from the two service calls.



Figure 11.2 With multiple services involved in a transaction, we see that they share the same trace ID.

In figure 11.2, we can see that both the licensing and organization services have the same trace ID, `85f4c6e4a1738e77`. However, the organization service has a span ID of `85f4c6e4a1738e77` (the same value as the transaction ID). The licensing service has a span ID of `382ce00e427adf7b`. By adding nothing more than a few .pom dependencies, we've replaced all the correlation ID infrastructure that you built in chapters 7 and 8.

11.2 Log aggregation and Spring Cloud Sleuth

In a large-scale microservice environment (especially in the cloud), logging data is a critical tool for debugging. However, because the functionality of a microservice-based application is decomposed into small, granular services, and we can have multiple service instances for a single service type, trying to tie debugging to log data from multiple ser-

vices to resolve a user's problem can be extremely difficult. Developers wanting to debug a problem across multiple servers often have to try the following:

- *Log into multiple servers to inspect the logs present on each server.* This is an extremely laborious task, especially if the services in question have different transaction volumes that cause logs to roll over at different rates.
- *Write home-grown query scripts that will attempt to parse the logs and identify the relevant log entries.* Because every query might be different, we often end up with a large proliferation of custom scripts for querying data from our logs.
- *Prolong the recovery of a downgraded service process to back up the logs residing on a server.* If a server hosting a service crashes completely, the logs are usually lost.

Each of the problems listed are real concerns that we often run into. Debugging a problem across distributed servers is ugly work and often significantly increases the amount of time it takes to identify and resolve an issue. A much better approach is to stream, real time, all the logs from all of our service instances to a centralized aggregation point, where the log data can be indexed and made searchable. Figure 11.3 shows at a conceptual level how this "unified" logging architecture would work.

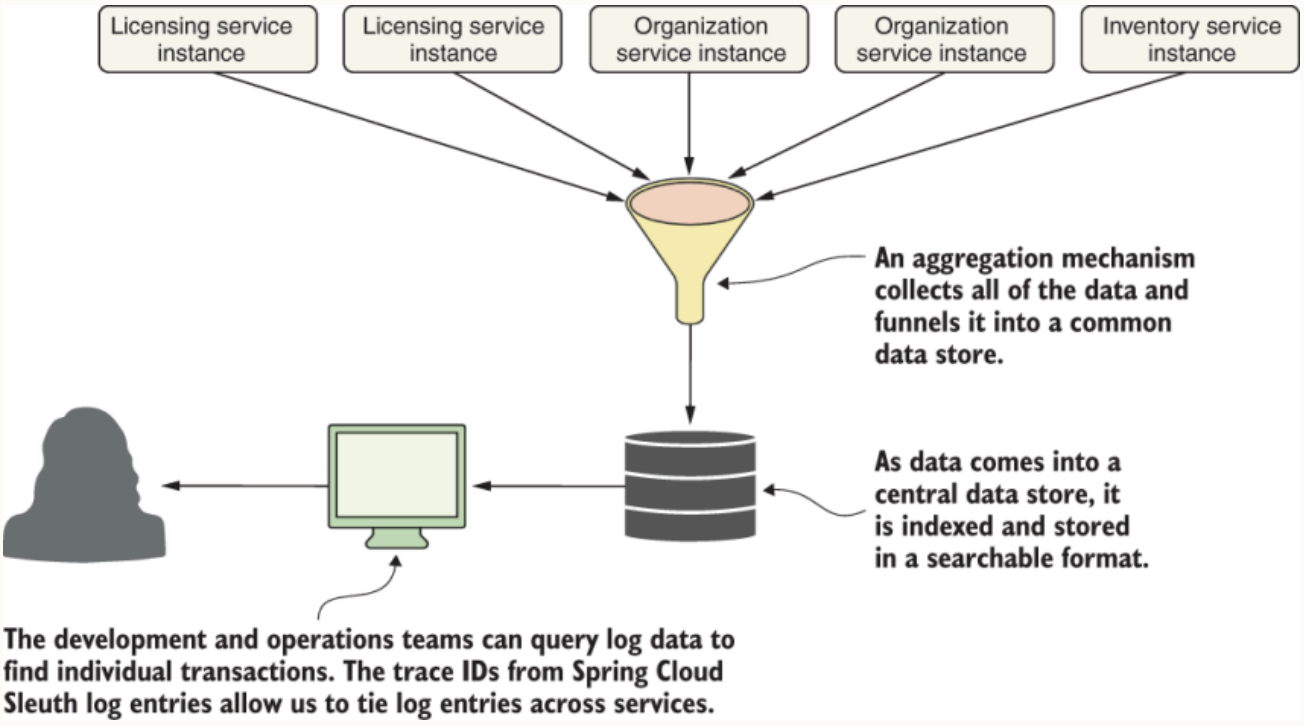


Figure 11.3 The combination of aggregated logs and a unique transaction ID across service log entries makes debugging distributed transactions more manageable.

Fortunately, there are multiple open source and commercial products that can help us implement the logging architecture in figure 11.3. Also, multiple implementation models exist that allow us to choose between an on-premises, locally managed solution or a

cloud-based solution. Table 11.1 summarizes several of the choices available for logging infrastructure.

Table 11.1 Log aggregation solutions for use with Spring Boot

Product name	Implementation models	Notes
Elasticsearch, Logstash, Kibana (the ELK Stack)	Commercial Open source	**https://www.elastic.co/what-is/elk-stack** General-purpose search engine Log aggregation through the ELK stack Typically implemented on premises
Graylog	Commercial Open source	**https://www.graylog.org/** Designed to be installed on premises
Splunk	Commercial	**https://www.splunk.com/** Oldest and most comprehensive of the log management and aggregation tools Initially an on-premises solution, now with a cloud offering
Sumo Logic	Commercial Freemium/tiered	**https://www.sumologic.com/** Runs only as a cloud service Requires a corporate work account to sign up (no Gmail or Yahoo accounts)
Papertrail	Commercial Freemium/tiered	**https://www.papertrail.com/** Runs only as a cloud service

With all these choices, it might be challenging to choose which one is the best. Every organization is going to be different and have different needs. For this chapter, we'll look at ELK as an example of how to integrate Spring Cloud Sleuth-backed logs into a unified logging platform. We've chosen the ELK Stack because:

- ELK is open source.
- It's straightforward to set up, simple to use, and user friendly.
- It's a complete tool that allows us to search, analyze, and visualize real-time logs generated from different services.

- It allows us to centralize all the logging to identify server and application issues.

.2.1 A Spring Cloud Sleuth/ELK Stack implementation in action

In figure 11.3, we saw a general unified logging architecture. Let's now see how we can implement the same architecture with Spring Cloud Sleuth and the ELK Stack. To set up ELK to work with our environment, we need to take the following actions:

1. Configure Logback in our services
2. Define and run the ELK Stack applications in Docker containers
3. Configure Kibana
4. Test the implementation by issuing queries based on the correlation IDs from Spring Cloud Sleuth

Figure 11.4 shows the end state for our implementation. In the figure, we can see how Spring Cloud Sleuth and ELK fit together for our solution.

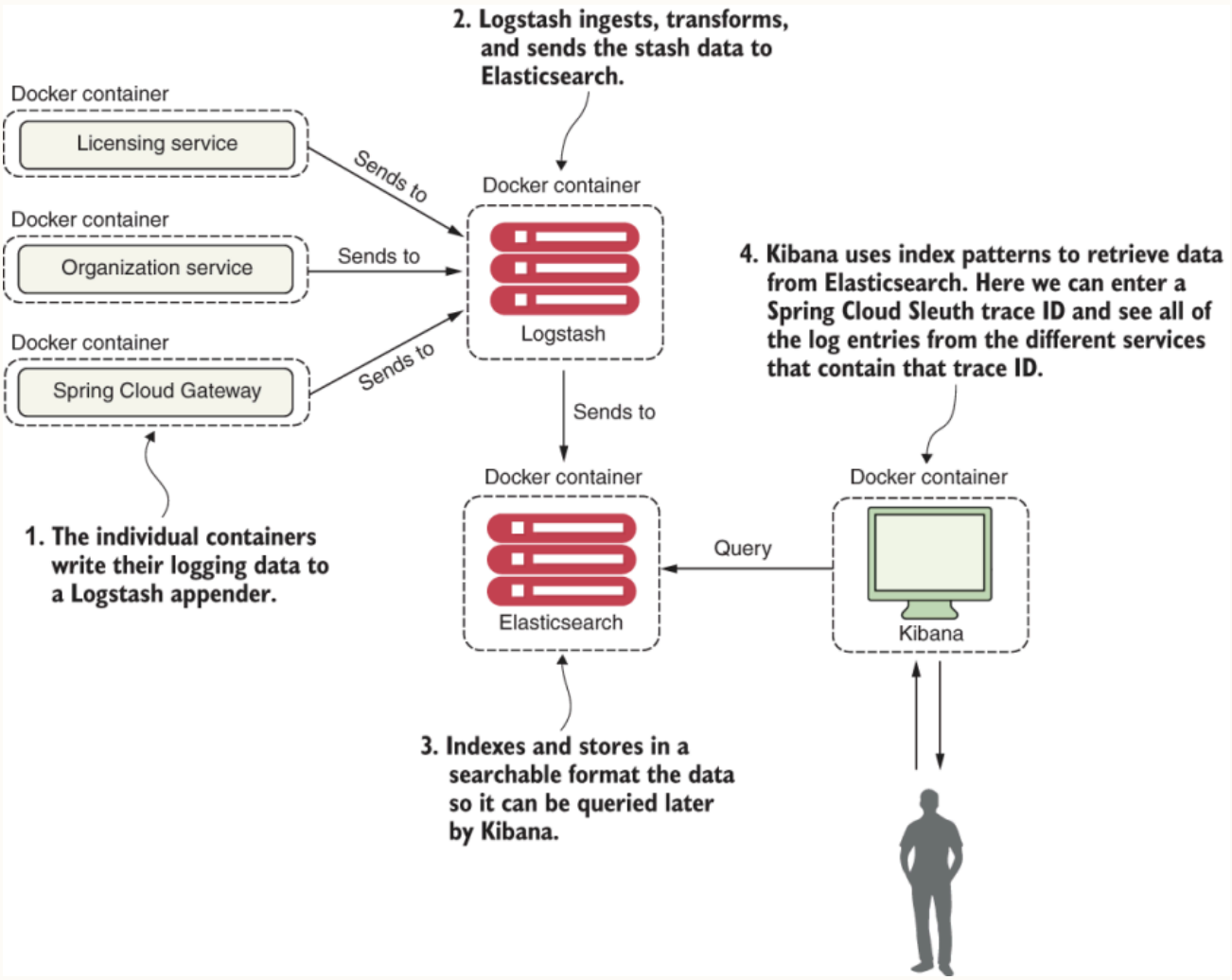


Figure 11.4 The ELK Stack allows us to quickly implement a unified logging architecture.

In figure 11.4, the licensing, organization, and gateway services communicate via TCP with Logstash to send the log data. Logstash filters, transforms, and passes the data to a

central data store (in this case, Elasticsearch). Elasticsearch indexes and stores the data in a searchable format so it can be queried later by Kibana. Once the data is stored, Kibana uses the index patterns from Elasticsearch to retrieve the data.

At this point, we can create a specific query index and enter a Spring Cloud Sleuth trace ID to see all of the log entries from the different services that contain it. Once the data is stored, we can look for the real-time logs by just accessing Kibana.

.2.2 Configuring Logback in our services

Now that we've seen the logging architecture with ELK, let's start configuring Logback for our services. In order to do this, we need to do the following:

1. Add the `logstash-logback-encoder` dependency in the pom.xml of our services
2. Create the Logstash TCP appender in a Logback configuration file

ADDING THE LOGSTASH ENCODER

To begin, we need to add the `logstash-logback-encoder` dependency to the pom.xml file of our licensing, organization, and gateway services. Remember, the pom.xml file can be found in the root directory of the source code. Here's the code to add the dependency:

```
<dependency>
   <groupId>net.logstash.logback</groupId>
   <artifactId>logstash-logback-encoder</artifactId>
   <version>6.3</version>
</dependency>
```

CREATING THE LOGSTASH TCP APPENDER

Once the dependency is added to each service, we need to tell the licensing service that it needs to communicate with Logstash to send the applications logs, formatted as JSON. (Logback, by default, produces the application logs in plain text, but to use Elasticsearch indexes, we need to make sure we send the log data in a JSON format.) There are three ways to accomplish this:

- Using the `net.logstash.logback.encoder.LogstashEncoder` class
- Using the `net.logstash.logback.encoder.LoggingEventCompositeJson-Encoder` class
- Parsing the plain-text log data with Logstash

For this example, we will use `LogstashEncoder`. We've chosen this class because it is the easiest and fastest to implement and because, in this example, we don't need to add additional fields to the logger. With `LoggingEventCompositeJsonEncoder`, we can add new patterns or fields, disable default providers, and more. If we choose one of these two classes, the one in charge of parsing the log files into a Logstash format is the Logback. With the third option, we can delegate parsing entirely to the Logstash using a JSON filter. All three options are good, but we suggest using the `LoggingEventCompositeJsonEncoder` when you have to add or remove default configurations. The other two options will depend entirely on your business needs.

NOTE You can choose whether to handle the log info in the application or in Logstash.

To configure this encoder, we'll create a Logback configuration file called logback-spring.xml. This configuration file should be located in the service resources folder. For the licensing service, the Logback configuration is shown in listing 11.1 and can be found in the licensing service's /licensing-service/src/main/resources/logback-spring.xml file. Figure 11.5 shows the log output that this configuration produces.

Listing 11.1 Configuring Logback with Logstash for the licensing service

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <include resource="org/springframework/boot/logging/logback/base.xml"/>
  <springProperty scope="context" name="application_name"
      source="spring.application.name"/>

  <appender name="logstash" class="net.logstash.logback.appender.
        LogstashTcpSocketAppender">                          (1)
    <destination>logstash:5000</destination>                    (2)
    <encoder class="net.logstash.logback.encoder.LogstashEncoder"/>
  </appender>

  <root level="INFO">
    <appender-ref ref="logstash"/>
    <appender-ref ref="CONSOLE"/>
  </root>
  <logger name="org.springframework" level="INFO"/>
  <logger name="com.optimagrowth" level="DEBUG"/>
</configuration>
```

(1) Indicates that we use TcpSocketAppender to communicate with Logstash

❷ Logstash hostname and port to establish the TCP communication


```
{
  "_index": "logstash-2020.05.30-000001",
  "_type": "_doc",
  "_id": "NwhPY3IBRu5zD4iyn8zO",
  "_version": 1,
  "_score": null,
  "_source": {
    "X-Span-Export": "false",
    "level_value": 10000,
    "@version": "1",
    "thread_name": "http-nio-8080-exec-9",
    "host": "licensing-service.docker_backend",
    "spanId": "6e761268be8708ed",
    "X-B3-TraceId": "6e761268be8708ed",
    "logger_name": "com.optimagrowth.license.service.client.OrganizationRestTemplateClient",
    "X-B3-SpanId": "6e761268be8708ed",
    "port": 51522,
    "spanExportable": "false",
    "message": "I have successfully retrieved an organization f31ced82-53e6-48d3-8969-0095ec7cdaf5 from the redis cache
    "@timestamp": "2020-05-30T02:01:02.043Z",
    "traceId": "6e761268be8708ed",
    "application_name": "licensing-service",
    "level": "DEBUG"
  },
  "fields": {
    "@timestamp": [
      "2020-05-30T02:01:02.043Z"
    ]
  },
  "sort": [
    1590804062043
  ]
}
```


Figure 11.5 Application log formatted with the LogstashEncoder

In figure 11.5, we can see two important aspects. The first is that `LogstashEncoder` includes all the values stored in Spring's MDC logger by default, and the second is that because we added the Spring Cloud Sleuth dependency to our service, we can see the `TraceId`, `X-B3-TraceId`, `SpanId`, `X-B3-SpanId`, and `spanExportable` fields in our log data. Note that the prefix *X-B3* propagates the default header that Spring Cloud Sleuth uses from service to service. This name consists of *X*, which is used for custom headers that are not part of the HTTP specification and *B3*, which stands for "BigBrotherBird," Zipkin's previous name.

NOTE If you want to know more about the MDC fields, we highly recommend that you read the SL4J logger documentation at **http://www.slf4j.org/manual.html#mdc** and the Spring Cloud Sleuth documentation at **https://cloud.spring.io/spring-cloud-static/spring-cloud-sleuth/2.1.0.RELEASE/single/spring-cloud-sleuth.html**.

You can also configure the log data shown in figure 11.5 by using the `LoggingEventCompositeJsonEncoder`. Using this composite encoder, we can disable all the providers that were added by default to our configuration, add new patterns to display custom or existent MDC fields, and more. The following listing shows a brief example of

the logback-spring.xml configuration that deletes some output fields and creates a new pattern with a custom field and other existent fields.

Listing 11.2 Customizing the Logback configuration for the licensing service

```
<encoder class="net.logstash.logback.encoder
          .LoggingEventCompositeJsonEncoder">
   <providers>
      <mdc>
         <excludeMdcKeyName>X-B3-TraceId</excludeMdcKeyName>
         <excludeMdcKeyName>X-B3-SpanId</excludeMdcKeyName>
         <excludeMdcKeyName>X-B3-ParentSpanId</excludeMdcKeyName>
      </mdc>
      <context/>
      <version/>
      <logLevel/>
      <loggerName/>
      <pattern>
         <pattern>
            <omitEmptyFields>true</omitEmptyFields>
            {
               "application": {
                  version: "1.0"
               },
               "trace": {
                  "trace_id": "%mdc{traceId}",
                  "span_id": "%mdc{spanId}",
                  "parent_span_id": "%mdc{X-B3-ParentSpanId}",
                  "exportable": "%mdc{spanExportable}"
               }
            }
         </pattern>
      </pattern>
      <threadName/>
      <message/>
      <logstashMarkers/>
      <arguments/>
      <stackTrace/>
   </providers>
</encoder>
```

Although we chose the `LogstashEncoder` option for this example, you should select the option that best fits your needs. Now that we have our Logback configuration in the li-

censing service, let's add the same configuration to our other services: configuration and gateway. We'll define and run the ELK Stack applications in Docker containers.

.2.3 Defining and running ELK Stack applications in Docker

To set up our ELK Stack containers, we need to follow two simple steps. The first is to create the Logstash configuration file, and the second is to define the ELK Stack applications in our Docker configuration. Before we start creating our configuration, though, it's important to note that the Logstash pipeline has two required and one optional element. The required elements are the inputs and outputs:

- *The input enables a specific source of events to be read by Logstash.* Logstash supports a variety of input plugins such as GitHub, Http, TCP, Kafka, and others.
- *The output is in charge of sending the event data to a particular destination.* Elastic supports a variety of plugins such as CSV, Elasticsearch, email, file, MongoDB, Redis, stdout, and others.

The optional element in the Logstash configuration is the filter plugins. These filters are in charge of performing intermediary processing on an event such as translations, adding new information, parsing dates, truncating fields, and so forth. Remember, Logstash ingests and transforms the log data received. Figure 11.6 describes the Logstash process.

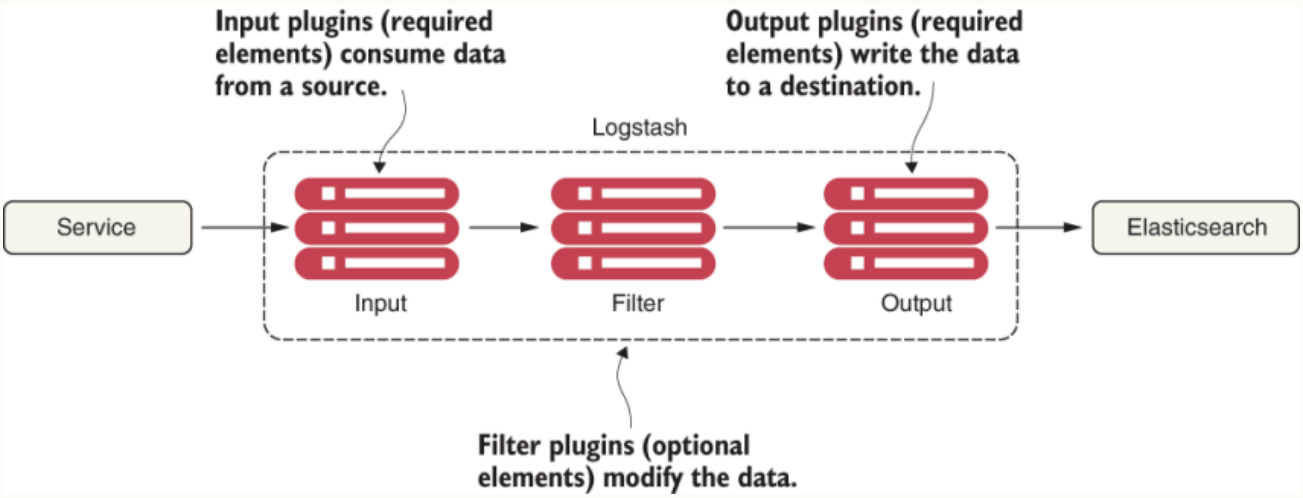


Figure 11.6 The Logstash configuration process contains two required (input and output) and one optional element (filter).

For this example, we'll use as the input plugin the Logback TCP appender that we previously configured and as the output plugin the Elasticsearch engine. The following listing shows the /docker/config/logstash.conf file.

Listing 11.3 Adding the Logstash configuration file

```
input {                          ❶
  tcp {
    port => 5000                 ❷
    codec => json_lines
  }
}

filter {
  mutate {
    add_tag => [ "manningPublications" ]    ❸
  }
}

output {                         ❹
  elasticsearch {
    hosts => "elasticsearch:9200"          ❺
  }
}
```

❶ TCP input plugin that reads events from a TCP socket

❷ Logstash port

❸ Mutate filter that adds a specific tag to the events

❹ Elasticsearch output plugin that sends the log data to the Elasticsearch engine

❺ Elasticsearch port

In listing 11.3, we can see five essential elements. The first is the `input` section. In this section, we specify the `tcp` plugin for consuming the log data. Next is the port number 5000; this is the port that we'll specify for Logstash later in the docker-compose.yml file. (If you take another look at figure 11.4, you'll notice that we are going to send the applications logs directly to Logstash.)

The third element is optional and corresponds to the filters; for this particular scenario, we added a mutate filter. This filter adds a `manningPublications` tag to the events. A real-world scenario of a possible tag for your services might be the environment where the application runs. Finally, the fourth and fifth elements specify the output plugin for our Logstash service and send the processed data to the Elasticsearch service running on port 9200. If you are interested in knowing more about all the input, output, and filter plugins Elastic offers, we highly recommend you visit the following pages:

- https://www.elastic.co/guide/en/logstash/current/input-plugins.html
- https://www.elastic.co/guide/en/logstash/current/output-plugins.html
- https://www.elastic.co/guide/en/logstash/current/filter-plugins.html

Now that we have the Logstash configuration, let's add the three ELK Docker entries to the docker-compose.yml file. Remember, we are using this file to fire up all of the Docker containers used for the code examples in this and the previous chapters. The following listing shows the docker/docker-compose.yml file with the new entries.

Listing 11.4 Configuring ELK Stack/Docker Compose

```
#Part of the docker-compose.yml removed for conciseness
#Some additional configuration also removed for conciseness

elasticsearch:
  image: docker.elastic.co/elasticsearch/
       elasticsearch:7.7.0                    (1)
  container_name: elasticsearch
  volumes:
   - esdata1:/usr/share/elasticsearch/data
  ports:
   - 9300:9300                                (2)
   - 9200:9200                                (3)
kibana:
  image: docker.elastic.co/kibana/kibana:7.7.0      (4)
  container_name: kibana
  environment:
   ELASTICSEARCH_URL: "http://elasticsearch:9300"   (5)
  ports:
   - 5601:5601                                (6)
logstash:
  image: docker.elastic.co/logstash/logstash:7.7.0  (7)
  container_name: logstash
  command:
   logstash -f /etc/logstash/conf.d/logstash.conf   (8)
  volumes:
   - ./config:/etc/logstash/conf.d.           (9)
  ports:
   - "5000:5000"                              (10)

#Rest of the docker-compose.yml omitted for conciseness
```

(1) Indicates the Elasticsearch image for our container (in this case, version 7.7.0)

❷ Maps the 9300 port as the cluster communication port

❸ Maps the 9200 port as the REST communication port

❹ Indicates the Kibana image that we'll use

❺ Sets the Elasticsearch URL and indicates the node/transport API 9300 port

❻ Maps the Kibana web application port

❼ Indicates the Logstash image that we'll use

❽ Loads the Logstash config from a specific file or directory

❾ Mounts the configuration file in a Logstash running container

❿ Maps the port for Logstash

NOTE Listing 11.4 contains a small part of the docker-compose.yml file for this chapter. In case you want to see the complete file, you can visit the following link: **https://github.com/ihuaylupo/manning-smia/tree/master/chapter11/docker**.

To run the Docker environment, we need to execute the following commands in the root directory where the parent pom.xml is located. The `mvn` command creates a new image with the changes we made for the organization, licensing, and gateway services:

```
mvn clean package dockerfile:build
docker-compose -f docker/docker-compose.yml up
```

NOTE If you see an error 137 exit code with `<container_name>` container on your console while executing the `docker-compose` command, visit the following link to increase the memory for Docker: **https://www.petefreitag.com/item/848.cfm**.

Now that we have our Docker environment set up and running, let's move on with the next step.

.2.4 Configuring Kibana

Configuring Kibana is a straightforward process, and we only need to configure it once. To access Kibana, open a web browser to the following link: http://localhost:5601/. The first time we access Kibana, a Welcome page is displayed. This page shows two options.

The first allows us to play with some sample data, and the second lets us explore the generated data from our services. Figure 11.7 shows Kibana's Welcome page.

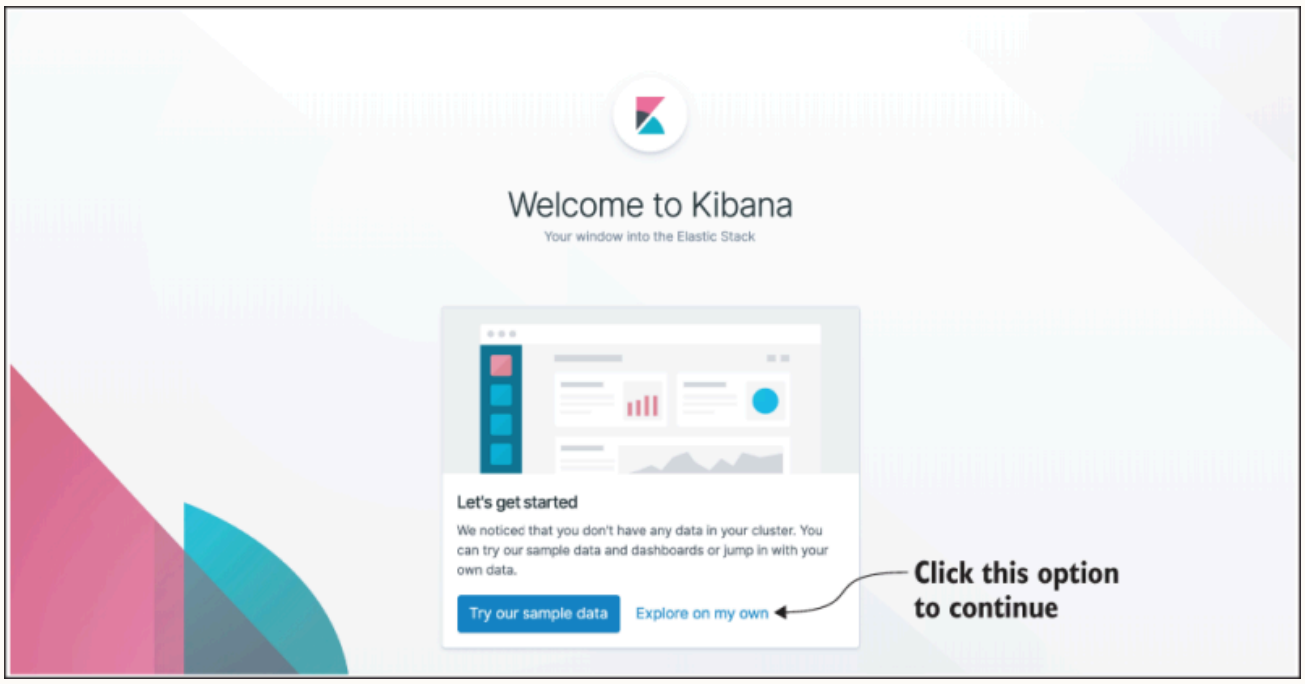


Figure 11.7 The Kibana Welcome page with two options: trying out the application with sample data or exploring it on your own.

To explore our data, let's click the Explore on My Own link. Once clicked, we will see an Add Data page like the one shown in figure 11.8. On this page, we need to click the Discover icon on the left side of the page.

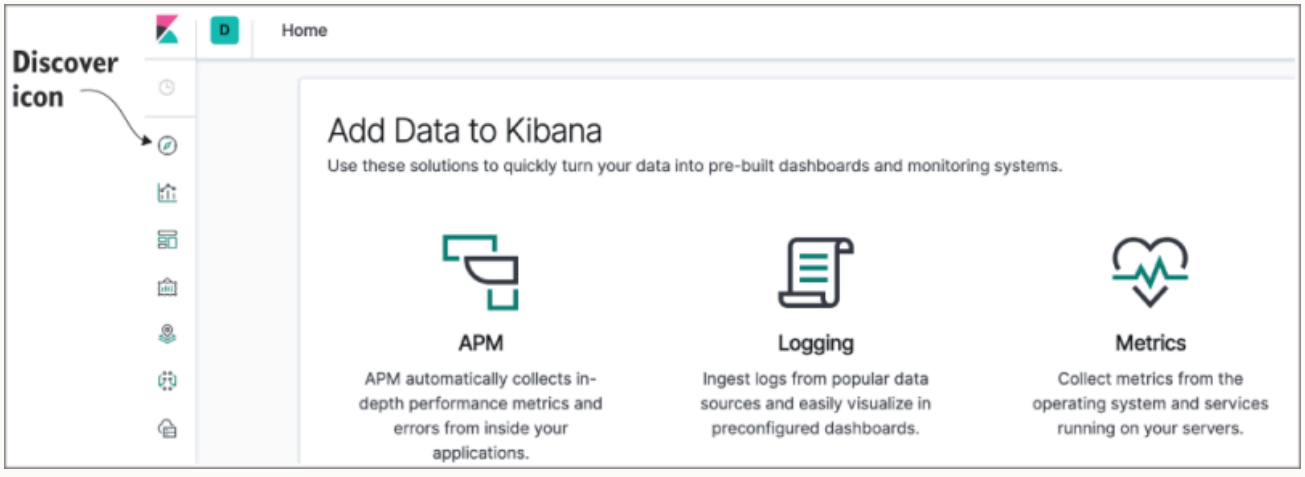


Figure 11.8 The Kibana setup page. Here we will see a set of options we can use to configure our Kibana application. Notice the icon menu on the left.

In order to continue, we must create an index pattern. Kibana uses a set of index patterns to retrieve the data from an Elasticsearch engine. The index pattern is in charge of telling Kibana which Elasticsearch indexes we want to explore. For example, in our case, we will create an index pattern indicating that we want to retrieve all of the Logstash in-

formation from Elasticsearch. To create our index pattern, click the Index Patterns link under the Kibana section at the left of the page. Figure 11.9 shows step 1 of this process.

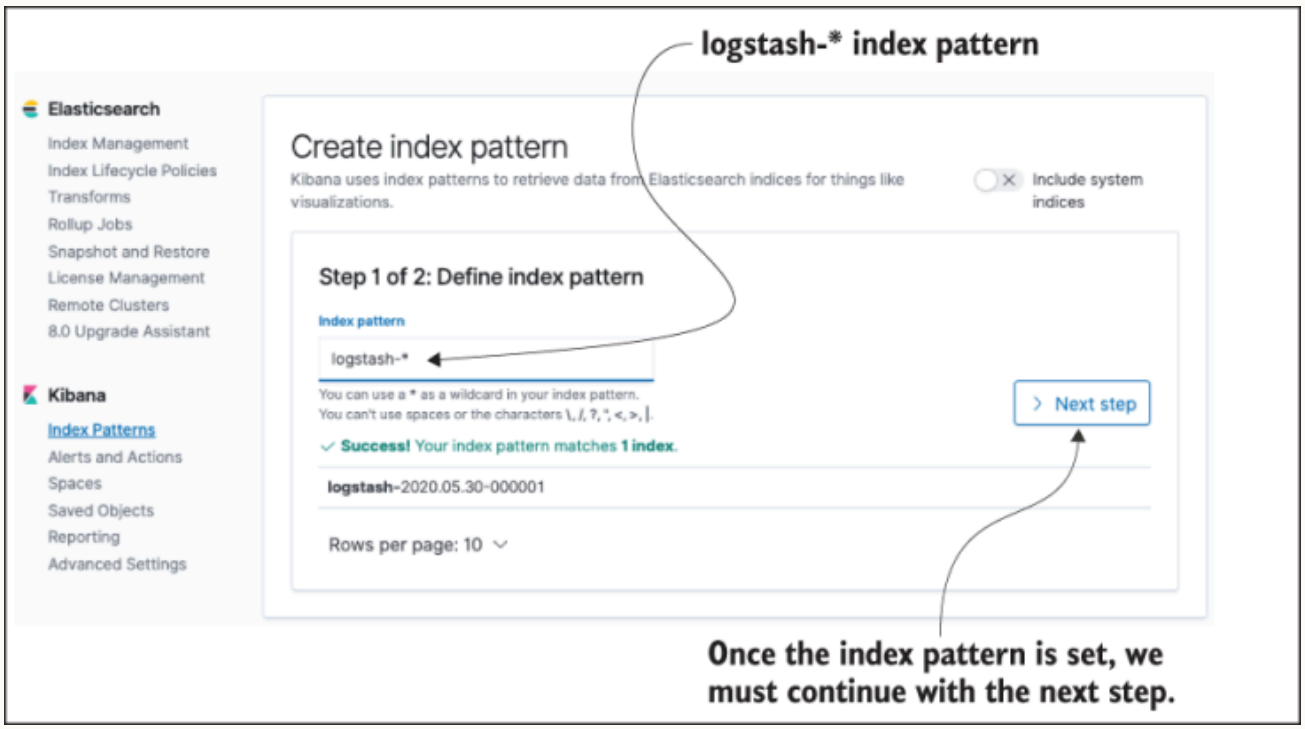


Figure 11.9 Configuring the index pattern for the Elasticsearch engine

On the Create Index Pattern page in figure 11.9, we can see that Logstash has already created an index as a first step. However, this index is not ready to use yet. To finish setting up the index, we must specify an index pattern for that index. To create it, we need to write the index pattern, `logstash-*`, and click the Next Step button.

For step 2, we'll specify a time filter. To do this, we need to select the @timestamp option under the Time Filter Field Name drop-down list and then click the Create Index Pattern button. Figure 11.10 shows this process.

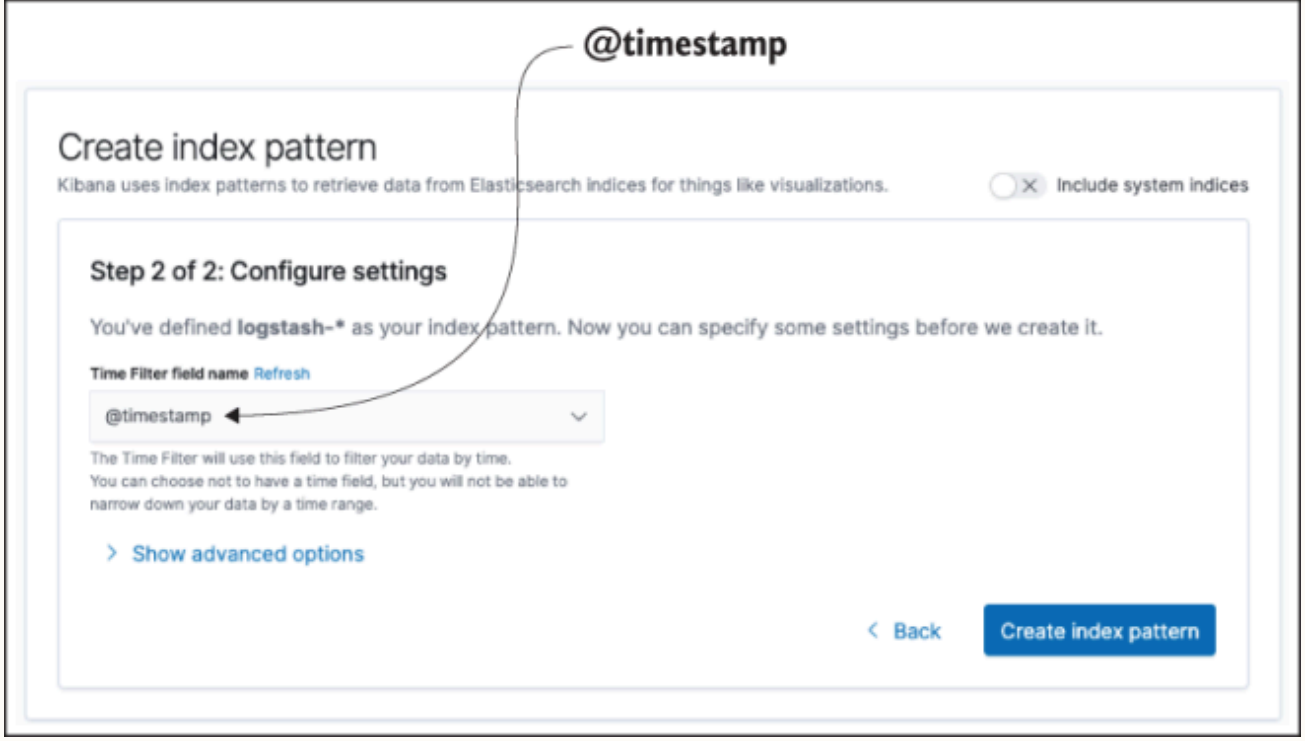


Figure 11.10 Configuring the timestamp filter for our index pattern. This pattern allows us to filter events within a time range.

We can now start making requests to our services to see the real-time logs in Kibana. Figure 11.11 shows an example of what the data sent to ELK should look like. If you don't see the page displayed in the figure, click the Discover icon again.



Figure 11.11 Individual service log events are stored, analyzed, and displayed by the ELK Stack.

At this point, we're all set up with Kibana. Let's continue with our final step.

.2.5 Searching for Spring Cloud Sleuth trace IDs in Kibana

Now that our logs are flowing to ELK, we can start to appreciate how Spring Cloud Sleuth adds trace IDs to our log entries. To query for all the log entries related to a single transaction, we need to take a trace ID and query it on Kibana's Discover screen (figure 11.12). By default, Kibana uses the Kibana Query Language (KQL), which is a simplified query syntax. When writing our query, you'll see that Kibana also provides a guide and autocomplete option to simplify the process of creating custom queries.

NOTE In order to apply the next filter, you need to select a valid trace ID. The trace ID used in the next example is not going to work in your Kibana instance.

Figure 11.12 shows how to execute a query with the Spring Cloud Sleuth trace ID. Here we use the trace ID `3ff985508b1b9365`.

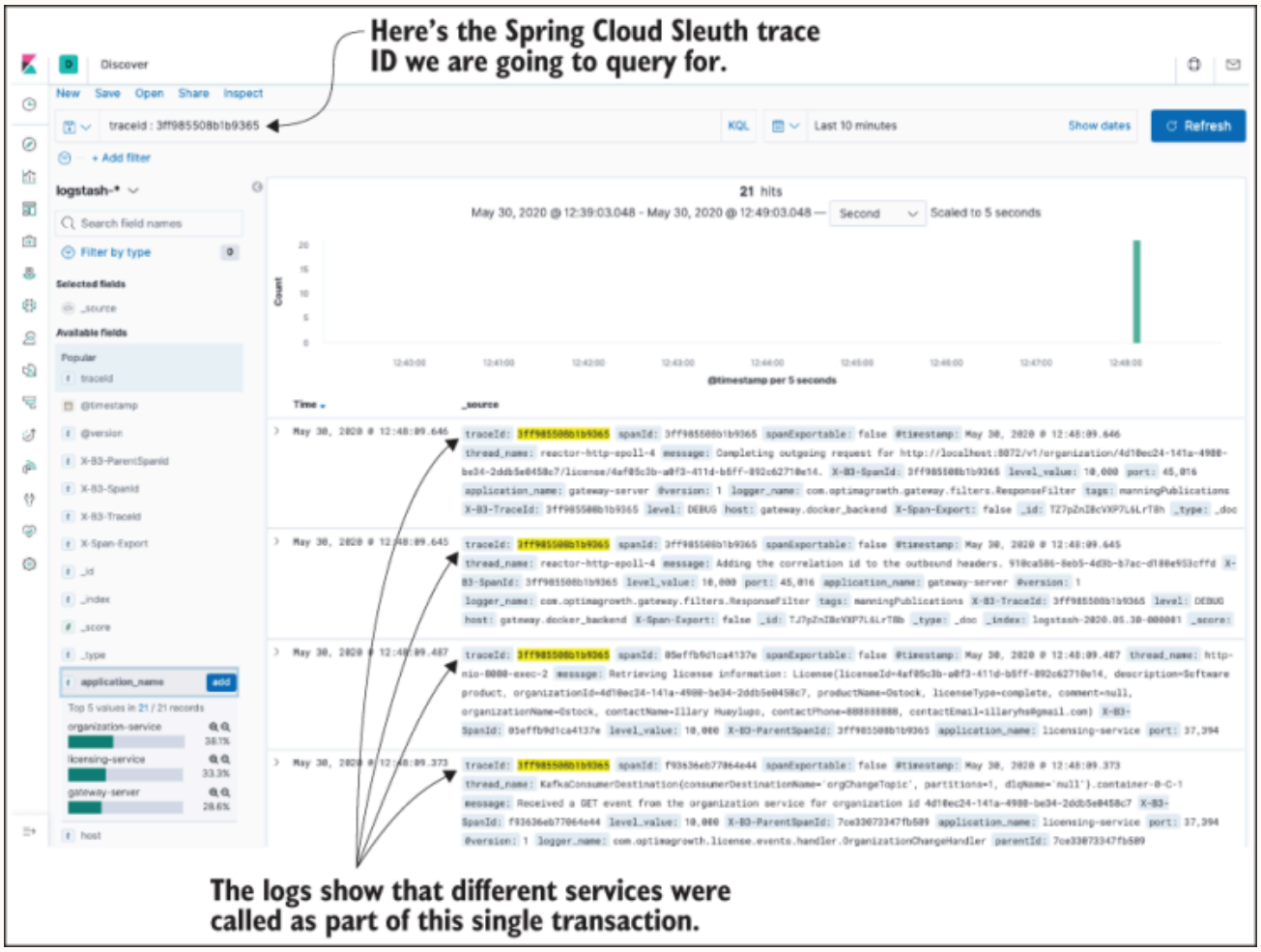


Figure 11.12 The trace ID allows you to filter all log entries that are related to a single transaction.

We can expand each log event if we want to see more details. By doing this, all the fields associated with a particular event will be displayed in a table or in JSON format. We will also be able to see all the additional information we added or transformed during the

Logstash processing. For example, we added a tag with the mutate filter in Logstash in listing 11.3. Figure 11.13 shows all the fields for this event.

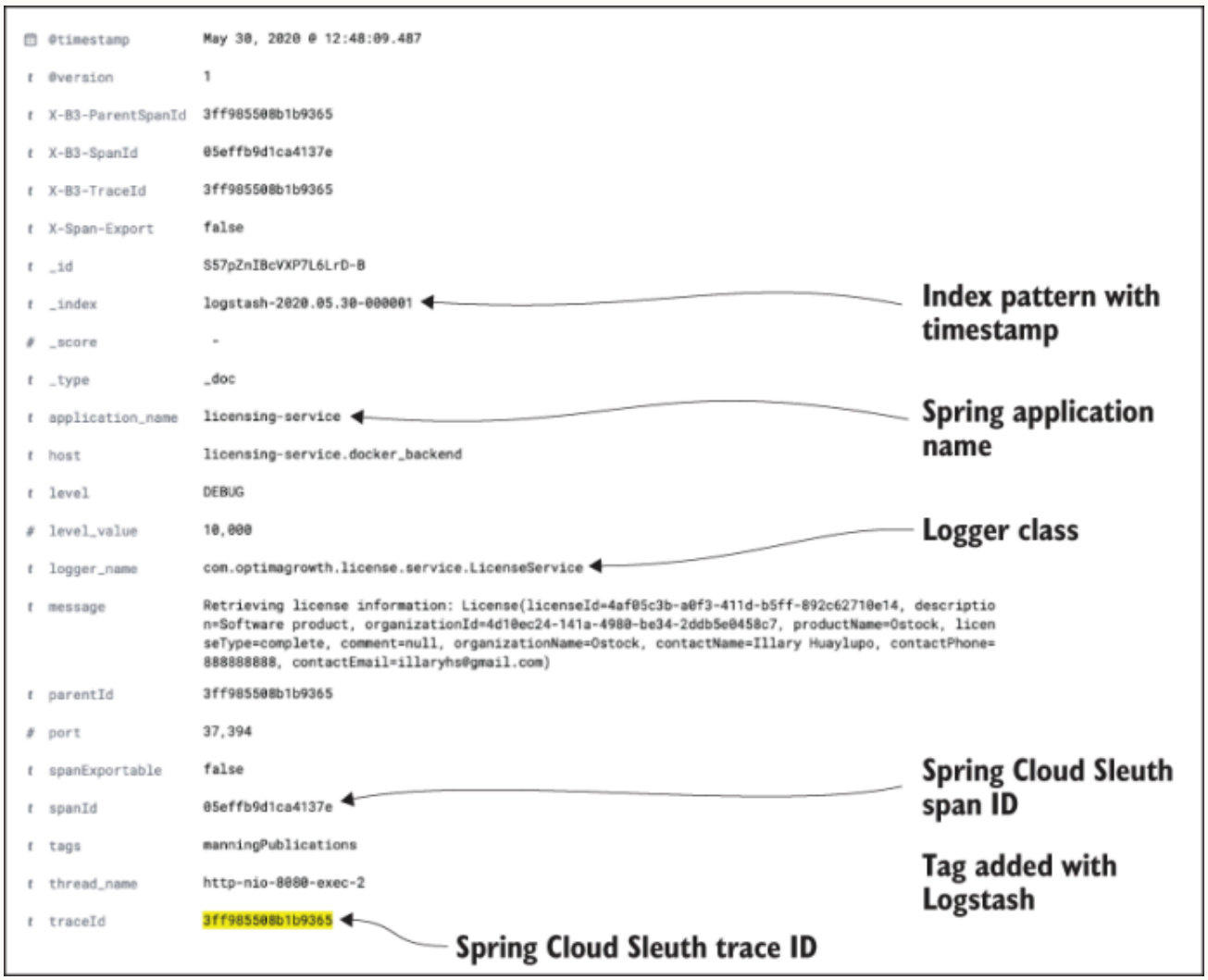


Figure 11.13 Detailed view of all the fields of a log event in Kibana

.2.6 Adding the correlation ID to the HTTP response with Spring Cloud Gateway

If we inspect the HTTP response from any service call made with Spring Cloud Sleuth, we'll see that the trace ID used in the call is never returned in the HTTP response headers. If we inspect the documentation for Spring Cloud Sleuth, we'll see that the Spring Cloud Sleuth team believes that returning any of the tracing data can be a potential security issue (though they don't explicitly list their reasons why they believe this). But we've found that returning a correlation or tracing ID in the HTTP response is invaluable when debugging a problem.

Spring Cloud Sleuth lets us "decorate" the HTTP response information with its tracing and span IDs. The process to do this, however, involves writing three classes and injecting two custom Spring beans. If you'd like to take this approach, you can see it in the Spring Cloud Sleuth documentation here:

- https://cloud.spring.io/spring-cloud-static/spring-cloud-sleuth/1.0.12.RELEASE/

A much simpler solution is to write a Spring Cloud Gateway filter that injects the trace ID into the HTTP response. In chapter 8, where we introduced the Spring Cloud Gateway, we saw how to build a gateway response filter, adding the generated correlation ID to the HTTP response returned by the caller for use in our services. Now we'll modify that filter to add a Spring Cloud Sleuth header. To set up our gateway response filter, we need to make sure that we have the Spring Cloud Sleuth dependencies in our pom.xml file like this:

```
<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-starter-sleuth</artifactId>
</dependency>
```

We'll use the `spring-cloud-starter-sleuth` dependency to tell Spring Cloud Sleuth that we want the gateway to participate in a Spring Cloud trace. Later in this chapter, when we introduce Zipkin, we'll see that the gateway service will be the first call in any service invocation. Once the dependency is in place, the actual response filter is trivial to implement.

The following listing shows the source code used to build the filter. The file is located in /gatewayserver/src/main/java/com/optimagrowth/gateway/filters/ResponseFilter .java .

Listing 11.5 Adding the Spring Cloud Sleuth trace ID via a response filter

```
package com.optimagrowth.gateway.filters;
//Imports removed for conciseness.
import brave.Tracer;
import reactor.core.publisher.Mono;

@Configuration
public class ResponseFilter {

  final Logger logger =LoggerFactory.getLogger(ResponseFilter.class);

  @Autowired
  Tracer tracer;                                    (1)

  @Autowired
  FilterUtils filterUtils;
```

```
    @Bean
    public GlobalFilter postGlobalFilter() {
        return (exchange, chain) -> {
            return chain.filter(exchange)
              .then(Mono.fromRunnable(() -> {
                String traceId =
                    Tracer.currentSpan()
                    .context()
                    .traceIdString();                    ❷
                logger.debug("Adding the correlation id to the outbound
                  headers. {}",traceId);
                exchange.getResponse().getHeaders()
                  .add(FilterUtils.CORRELATION_ID,traceId);
                logger.debug("Completing outgoing request for {}.",
                  exchange.getRequest().getURI());
              }));
        };
    }
}
```

❶ Sets the entry point to access trace and span ID information

❷ Adds a span to the HTTP response header tmx-correlation-ID in the Spring Cloud Sleuth trace ID

Because the gateway is now Spring Cloud Sleuth–enabled, we can access tracing information from within our `ResponseFilter` by autowiring in the `Tracer` class. This class lets us access information about the currently executed trace. The `tracer.currentSpan().context().traceIdString()` method enables us to retrieve the current trace ID as a String for the transaction underway. It's trivial to add the trace ID to the outgoing HTTP response passing back through the gateway. This is done with the following method call:

```
exchange.getResponse().getHeaders()
      .add(FilterUtils.CORRELATION_ID, traceId);
```

With this code in place, if we invoke an O-stock microservice through our gateway, we should get an HTTP response called `tmx-correlation-id` with a Spring Cloud Sleuth trace ID. Figure 11.14 shows the results of a call to the following endpoint to GET the licenses of an organization:

```
http://localhost:8072/license/v1/organization/4d10ec24-141a-4980-be34-
➥ 2ddb5e0458c7/license/4af05c3b-a0f3-411d-b5ff-892c62710e14
```

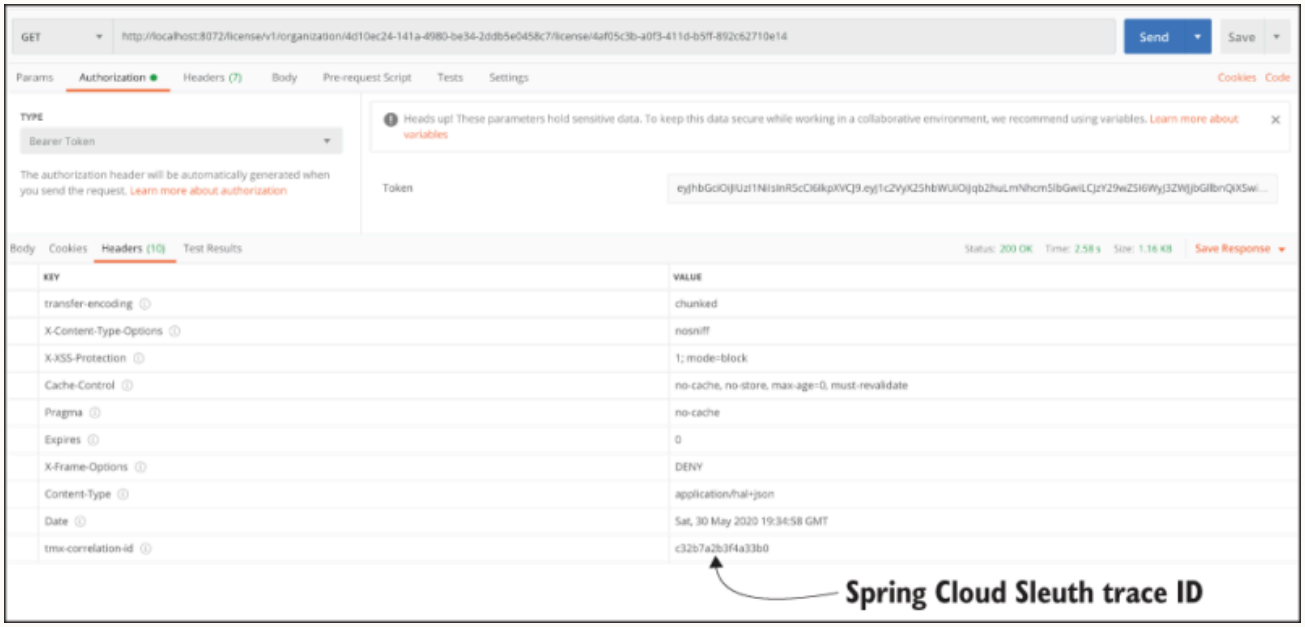


Figure 11.14 With the Spring Cloud Sleuth trace ID returned, we can easily query Kibana for the logs.

11.3 Distributed tracing with Zipkin

Having a unified logging platform with correlation IDs is a powerful debugging tool. However, for the rest of the chapter, we'll move away from tracing log entries and look at how to visualize the flow of transactions as they move across different microservices instead. A bright, concise picture is worth more than a million log entries.

Distributed tracing involves providing a visual picture of how a transaction flows across our different microservices. Distributed tracing tools also give a rough approximation of individual microservice response times. However, distributed tracing tools shouldn't be confused with full-blown Application Performance Management (APM) packages. APM packages offer out-of-the-box, low-level performance data on the actual service code, as well as performance data such as memory, CPU utilization, and I/O utilization beyond the response time.

This is where the Spring Cloud Sleuth and Zipkin (also referred to as OpenZipkin) projects shine. Zipkin (**http://zipkin.io/**) is a distributed tracing platform that allows us to trace transactions across multiple service invocations. It lets us graphically see the amount of time a transaction takes and breaks down the time spent in each microservice involved in the call. Zipkin is an invaluable tool for identifying performance issues in a microservices architecture. Setting up Spring Cloud Sleuth and Zipkin involves

- Adding Spring Cloud Sleuth and Zipkin JAR files to the services that capture trace data
- Configuring a Spring property in each service to point to the Zipkin server that will collect the trace data
- Installing and configuring a Zipkin server to collect the data
- Defining the sampling strategy each client will use to send tracing information to Zipkin

.3.1 Setting up the Spring Cloud Sleuth and Zipkin dependencies

We've included the Spring Cloud Sleuth dependencies in our licensing and organization services. These JAR files now include the necessary Spring Cloud Sleuth libraries to enable Spring Cloud Sleuth within a service. We next need to include a new dependency, the `spring-cloud-sleuth-zipkin` dependency, to integrate with Zipkin. The following listing shows the Maven entry that should be present in the Spring Cloud Gateway, licensing, and organization services.

Listing 11.6 Client-side Spring Cloud Sleuth and Zipkin dependencies

```
<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-sleuth-zipkin</artifactId>
<dependency>
```

.3.2 Configuring the services to point to Zipkin

With the JAR files in place, we need to configure each service that wants to communicate with Zipkin. We'll do this by setting a Spring property that defines the URL used to communicate with Zipkin. The property that needs to be set is the `spring.zipkin.baseUrl` property. This property is set in each service's configuration file located in the repository of the Spring Cloud Config Server (for example, the /configserver/src/main/resources/config/licensing-service.properties file for the licensing service). To run it locally, set the value of the `baseUrl` property to `localhost:9411` . However, if you want to run it with Docker, you need to override that value with `zipkin:9411` like this:

```
zipkin.baseUrl: zipkin:9411
```

Zipkin, RabbitMQ, and Kafka

Zipkin has the ability to send its tracing data to a Zipkin server via RabbitMQ or Kafka. From a functionality perspective, there's no difference in Zipkin's behavior if we use HTTP, RabbitMQ, or Kafka. With HTTP tracing, Zipkin uses an asynchronous thread to send performance data. The main advantage of using RabbitMQ or Kafka to collect tracing data is that if our Zipkin server is down, tracing messages sent to Zipkin are "enqueued" until Zipkin can pick up the data. Configuring Spring Cloud Sleuth to send data to Zipkin via RabbitMQ and Kafka is covered in the Spring Cloud Sleuth documentation, so we won't cover it here in further detail.

.3.3 Configuring a Zipkin server

There are several ways to set up Zipkin, but we'll use a Docker container with the Zipkin server. This option will allow us to avoid the creation of a new project in our architecture. To set up Zipkin, we'll add the following registry entry to our docker-compose.yml file located in the Docker folder for the project:

```
zipkin:
   image: openzipkin/zipkin
   container_name: zipkin
   ports:
     - 9411: 9411
   networks:
     backend:
       aliases:
         - "zipkin"
```

To run a Zipkin server, little configuration is needed. One of the few things we need to configure when we run Zipkin is the backend data store that Zipkin uses to store the tracing data. Zipkin supports four different backend data stores:

- In-memory data
- MySQL (**http://mysql.com/**)
- Cassandra (**https://cassandra.apache.org/**)
- Elasticsearch (**http://elastic.co/**)

By default, Zipkin uses an in-memory data store for storage. The Zipkin team, however, advises against using the in-memory database in a production environment. The in-memory database holds a limited amount of data, and the data is lost when the Zipkin server is shut down or fails.

For this example, we'll show you how to use Elasticsearch as a data store because we've already configured Elasticsearch. The only additional settings we need to add are the `STORAGE_TYPE` and `ES_HOSTS` variables in the `environment` section of our configuration file. The following code shows the complete Docker Compose registry:

```
zipkin:
    image: openzipkin/zipkin
    container_name: zipkin
    depends_on:
      - elasticsearch
    environment:
      - STORAGE_TYPE=elasticsearch
      - "ES_HOSTS=elasticsearch:9300"
    ports:
      - "9411:9411"
    networks:
      backend:
        aliases:
          - "zipkin"
```

.3.4 Setting tracing levels

Now we have the clients configured to talk to a Zipkin server, and we have the server configured and ready to run. But we need to do one more step before we can start using Zipkin and that is to define how often each service should write data to Zipkin.

By default, Zipkin only writes 10% of all transactions to the Zipkin server. The default value ensures that Zipkin will not overwhelm our logging and analysis infrastructure. The transaction sampling can be controlled by setting a Spring property on each of the services sending data to Zipkin: the `spring.sleuth.sampler.percentage` property. This property takes a value between 0 and 1 as follows:

- A value of 0 means Spring Cloud Sleuth doesn't send Zipkin any transactions.
- A value of .5 means Spring Cloud Sleuth sends 50% of all transactions.
- A value of 1 means Spring Cloud Sleuth sends all transactions (100%).

For our purposes, we'll send all trace information (100%) for our services and all transactions. To do this, we can set the value of `spring.sleuth.sampler.percentage`, or we can replace the default `Sampler` class used in Spring Cloud Sleuth with `AlwaysSampler`. The class `AlwaysSampler` can be injected as a Spring bean into an application. But for

this example, we will use the `spring.sleuth.sampler.percentage` in the configuration file of the licensing, organization, and gateway services as follows:

```
zipkin.baseUrl: zipkin:9411
spring.sleuth.sampler.percentage: 1
```

.3.5 Using Zipkin to trace transactions

Let's start this section with a scenario. Imagine you're one of the developers on the O-stock application, and you're on call this week. You get a support ticket from a customer who's complaining that one of the screens in the application is running slow. You have a suspicion that the licensing service being used by the screen is the culprit. But why and where?

In our scenario, the licensing service relies on the organization service, and both services make calls to different databases. Which service is the poor performer? Also, you know that these services are continually being modified, so someone might have added a new service call to the mix.

NOTE Understanding all the services that participate in the user's transaction and their individual performance times is critical in supporting a distributed architecture such as a microservices architecture.

To solve this dilemma, we'll use Zipkin to watch two transactions from our organization service as the Zipkin service traces them. The organization service is a simple service that only makes a call to a single database. What we're going to do is to use Postman to send two calls to the organization service with the following endpoint. The organization service calls will flow through the gateway before the calls get directed downstream to an organization service instance:

```
GET http://localhost:8072/organization/v1/organization/4d10ec24-141a-4980-
➥ be34-2ddb5e0458c6
```

If we look at the screenshot in figure 11.15, we'll see that Zipkin captured two transactions and that each transaction is broken down into one or more spans. In Zipkin, a *span* represents a specific service or call in which timing information is captured. Each of the transactions in figure 11.15 has five spans: two spans in the gateway, two for the organization, and one for the licensing service.

Remember, the gateway doesn't blindly forward an HTTP call. It receives the incoming HTTP call, terminates it, and then builds a new call to send to the targeted service (in this case, the organization service). The termination of the original call is how the gateway can add response, pre-, and post-filters to each call entering the gateway. It's also why we see two spans in the gateway service in figure 11.15.

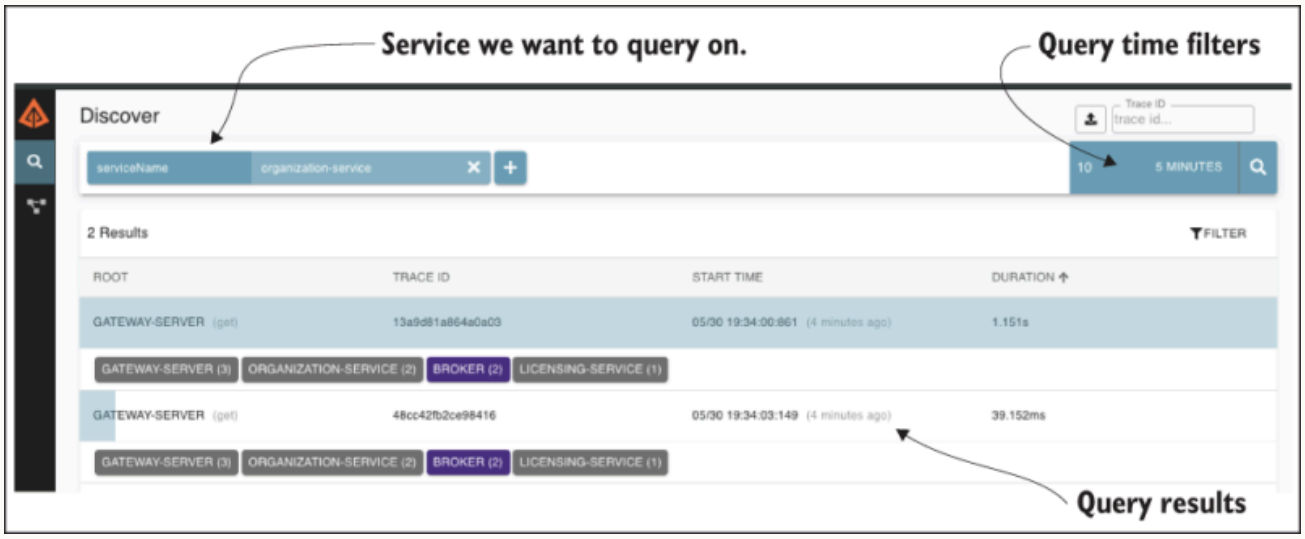


Figure 11.15 The Zipkin query screen, where we can select the service we want to trace, along with some basic query filters

The two calls to the organization service through the gateway took 1.151 seconds and 39.152 ms, respectively. Let's dig into the details of the longest-running call (1.151 seconds). We can see more detail by clicking the transaction and drilling down. Figure 11.16 shows the details for our calls to the organization service.

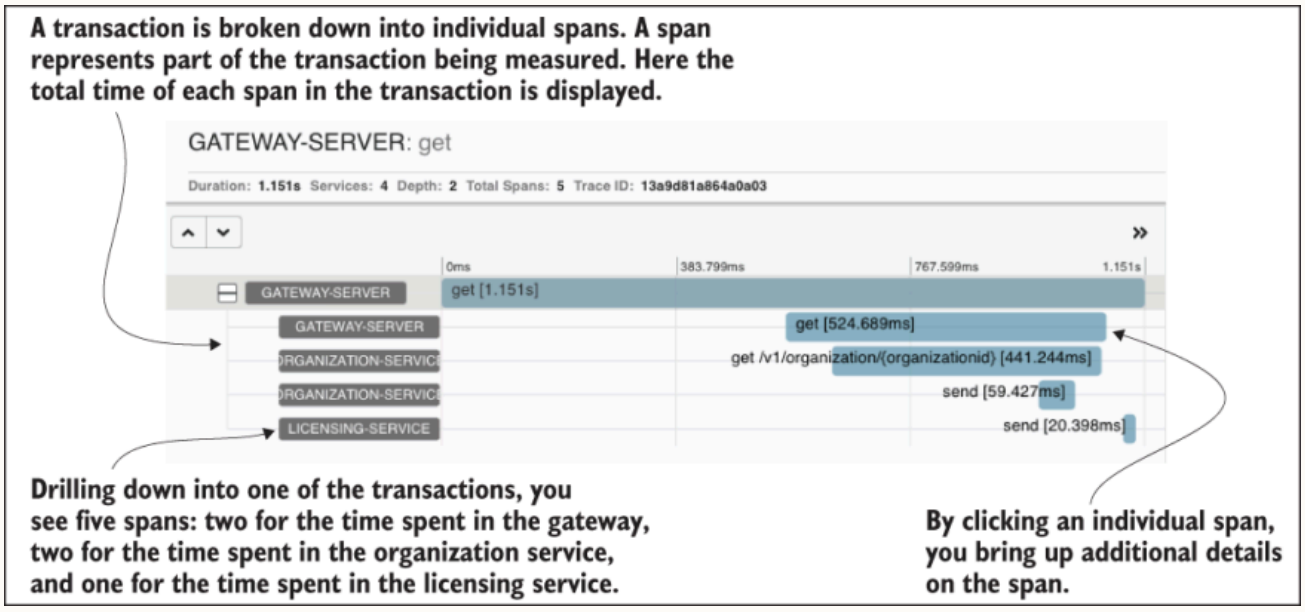


Figure 11.16 Zipkin allows us to drill down and see the amount of time each span takes with a transaction.

In figure 11.16, we can see that the entire transaction from a gateway perspective took approximately 1.151 seconds. However, the organization service call made by the gateway took 524.689 ms of the 1.151 seconds involved in the overall call. Let's drill down in each span for even more detail. Click the organization-service span and notice the additional details from the call (figure 11.17).

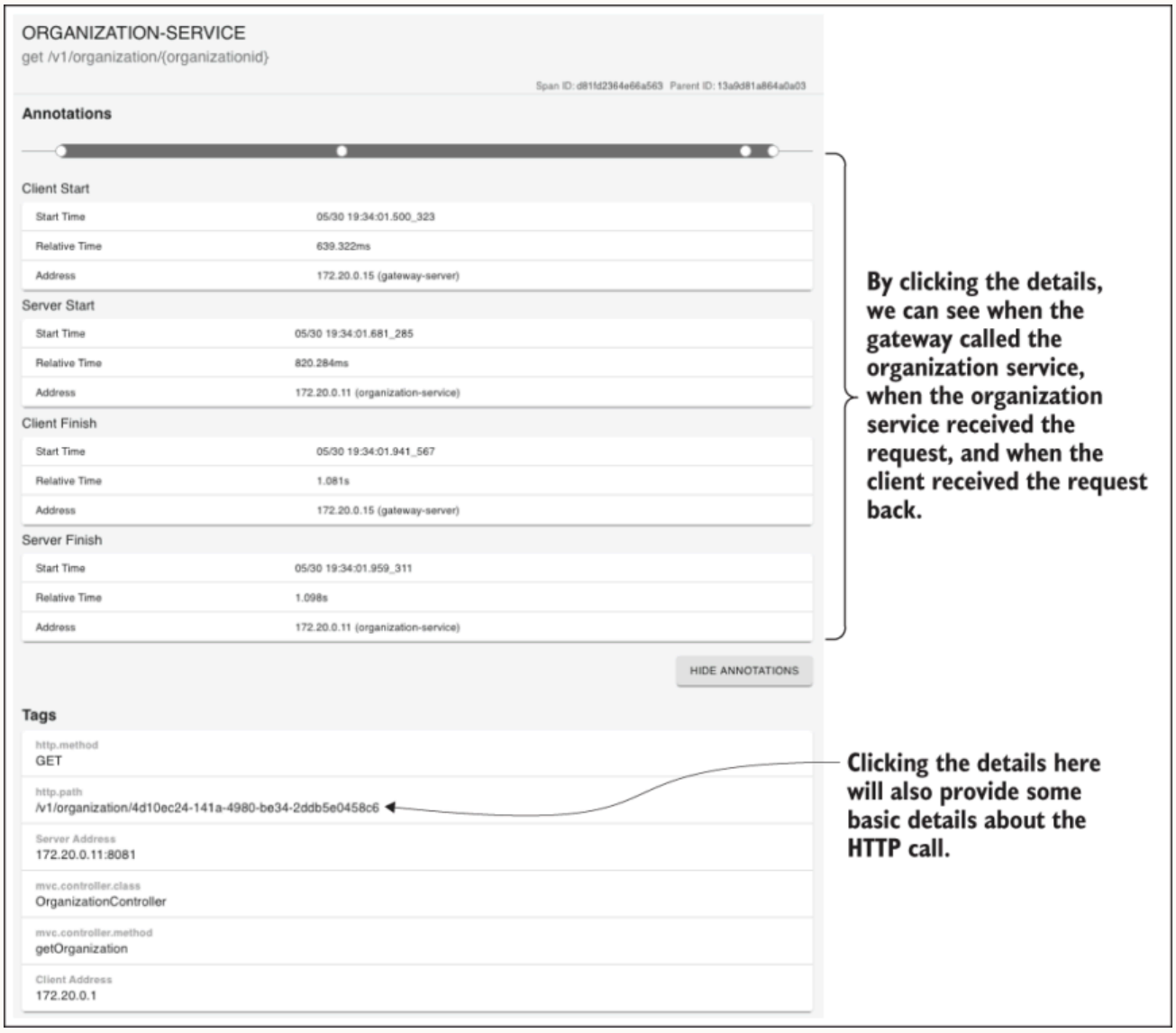


Figure 11.17 Clicking an individual span provides further details on the HTTP call and timing.

One of the most valuable pieces of information in figure 11.17 is the breakdown of the client (Gateway) as it called the organization service, when the organization service received the call, and when the organization service responded. This type of timing information is invaluable in detecting and identifying network latency issues. To add a custom span to the licensing service's call to Redis, we'll use the following class:

/licensing-service/src/main/java/com/optimagrowth/license/service/client/
↪ OrganizationRestTemplateClient.java

With the `OrganizationRestTemplateClient`, we'll implement the `checkRedisCache()` method. The following listing shows this code.

Listing 11.7 Adding `checkRedisCache()` to `OrganizationRestTemplate`

```
package com.optimagrowth.license.service.client;
//Rest of imports omitted for conciseness

@Component
public class OrganizationRestTemplateClient {
  @Autowired
  RestTemplate restTemplate;

  @Autowired
  Tracer tracer;                                          (1)

  @Autowired
  OrganizationRedisRepository redisRepository;

  private static final Logger logger =
      LoggerFactory.getLogger(OrganizationRestTemplateClient.class);

  private Organization checkRedisCache                    (2)
        (String organizationId) {
    try {
      return redisRepository.findById(organizationId).orElse(null);
    } catch (Exception ex){
       logger.error("Error encountered while trying to retrieve
           organization {} check Redis Cache. Exception {}",
           organizationId, ex);
       return null;
    }
  }

  //Rest of class omitted for conciseness
}
```

(1) Tracer accesses the Spring Cloud Sleuth trace information.

(2) Implements the CheckRedisCache method

.3.6 Visualizing a more complex transaction

What if we want to understand precisely which service dependencies exist between service calls? We can call the licensing service through the gateway and then query Zipkin for licensing service traces with a GET call to the licensing services using the following endpoint. Figure 11.18 shows the detailed trace of this call.

```
http://localhost:8072/license/v1/organization/4d10ec24-141a-4980-be34-
➥ 2ddb5e0458c8/license/4af05c3b-a0f3-411d-b5ff-892c62710e15
```

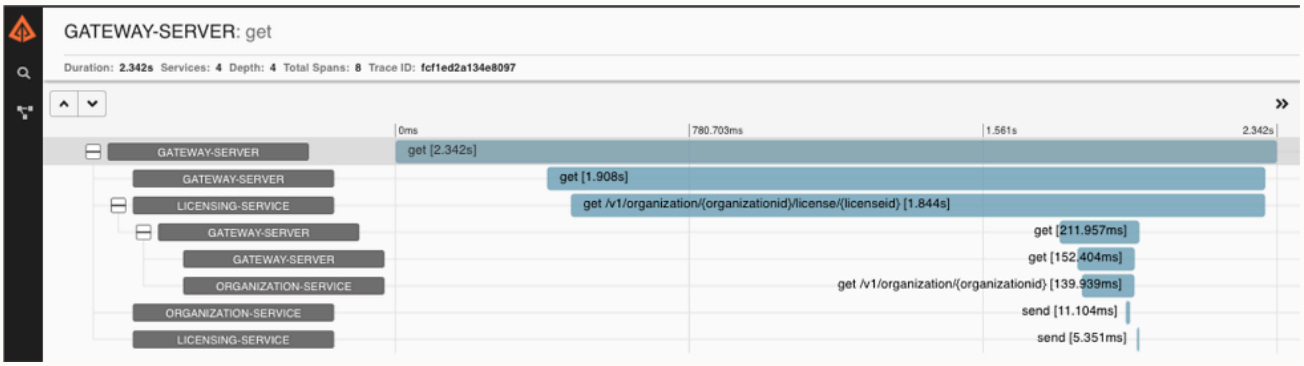


Figure 11.18 Viewing the details of a trace that shows how a licensing service call flows from the gateway to the licensing service and then to the organization service.

In figure 11.18, we can see that the call to the licensing service involves eight HTTP calls. We see the call to the gateway and then from the gateway to the licensing service; the licensing service to the gateway and the gateway to the organization; and finally, the organization to the licensing service using Apache Kafka to update the Redis cache.

.3.7 Capturing messaging traces

Messaging can introduce its own performance and latency issues inside an application. A service might not be processing a message from a queue quickly enough. Or there could be a network latency problem. We've encountered these scenarios while building microservice-based applications.

Spring Cloud Sleuth sends Zipkin trace data on any inbound or outbound message channel registered in the service. Using Spring Cloud Sleuth and Zipkin, we can identify when a message is published from a queue and when it's received. We can also see what behavior takes place when the message is received on a queue and processed. And, you'll remember from chapter 10, when an organization record is added, updated, or deleted, a Kafka message is produced and published via Spring Cloud Stream. The licensing service receives the message and updates a Redis key-value store that it uses to cache the data.

Next we'll delete an organization record and watch Spring Cloud Sleuth and Zipkin trace the transaction. We'll issue a DELETE to the following endpoint for the organization service via Postman:

```
http://localhost:8072/organization/v1/organization/4d10ec24-141a-4980-be34-
↪ 2ddb5e0458c7
```

Earlier in the chapter, we saw how to add the trace ID as an HTTP response header, adding a new HTTP response header called `tmx-correlation-id`. We had the `tmx-correlation-id` returned on our call with a value of `054accff01c9ba6b`. We can search Zipkin for this specific trace by entering the trace ID returned by our call into the search box in the upper-right corner of the Zipkin query screen. Figure 11.19 shows where we can enter the trace ID.

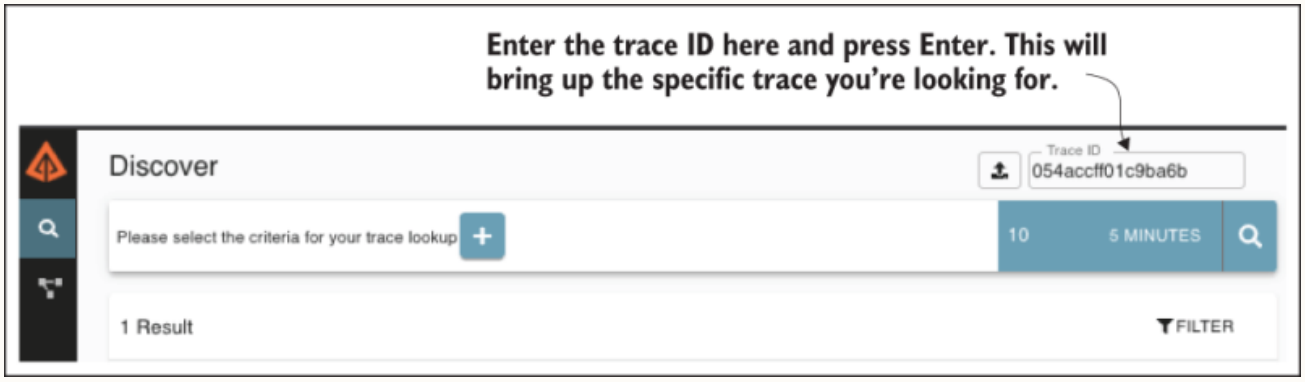


Figure 11.19 With the trace ID returned in the HTTP `tmx-correlation-id` response header, we can easily find the transaction by searching on the specific trace ID.

With the specific trace in hand, we can query Zipkin for the transaction and view the publication of the DELETE message. The second span in figure 11.20 is the output message from the channel output, which is used to publish to a Kafka topic called `orgChangeTopic`. Figure 11.20 shows the output message channel and how it appears in the Zipkin trace.

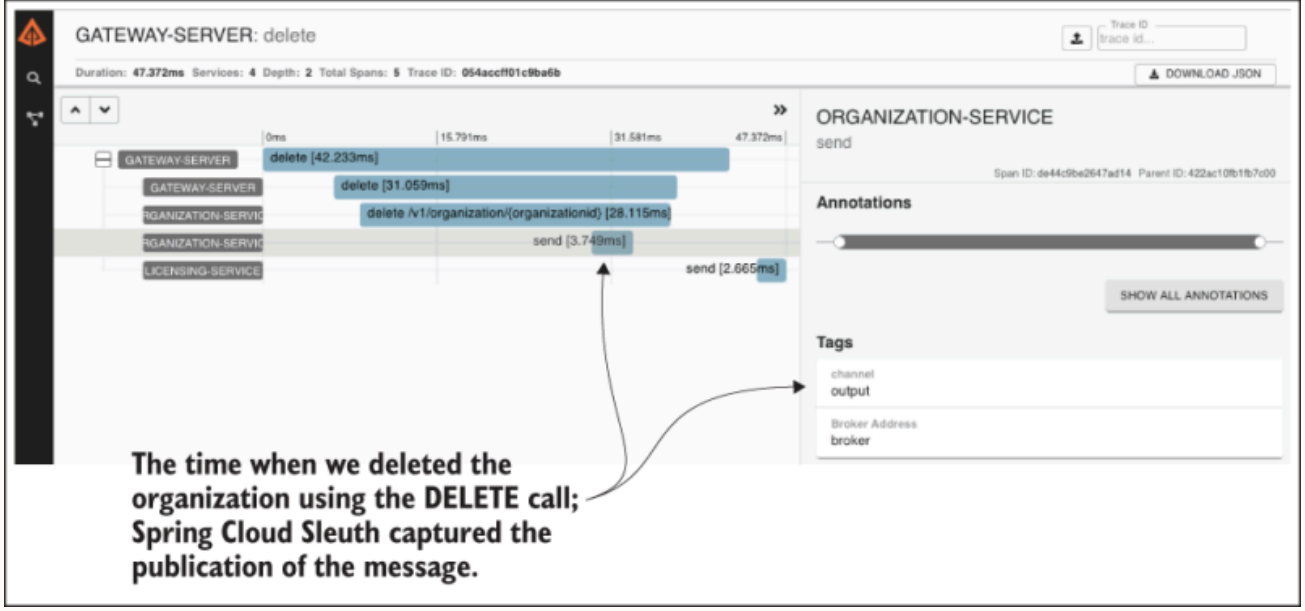


Figure 11.20 Spring Cloud Sleuth automatically traces the publication and receipt of messages on the Spring message channels. We can view the details of the trace with Zipkin.

You can see the licensing service receive the message by clicking the licensing service span. Figure 11.21 shows this data.

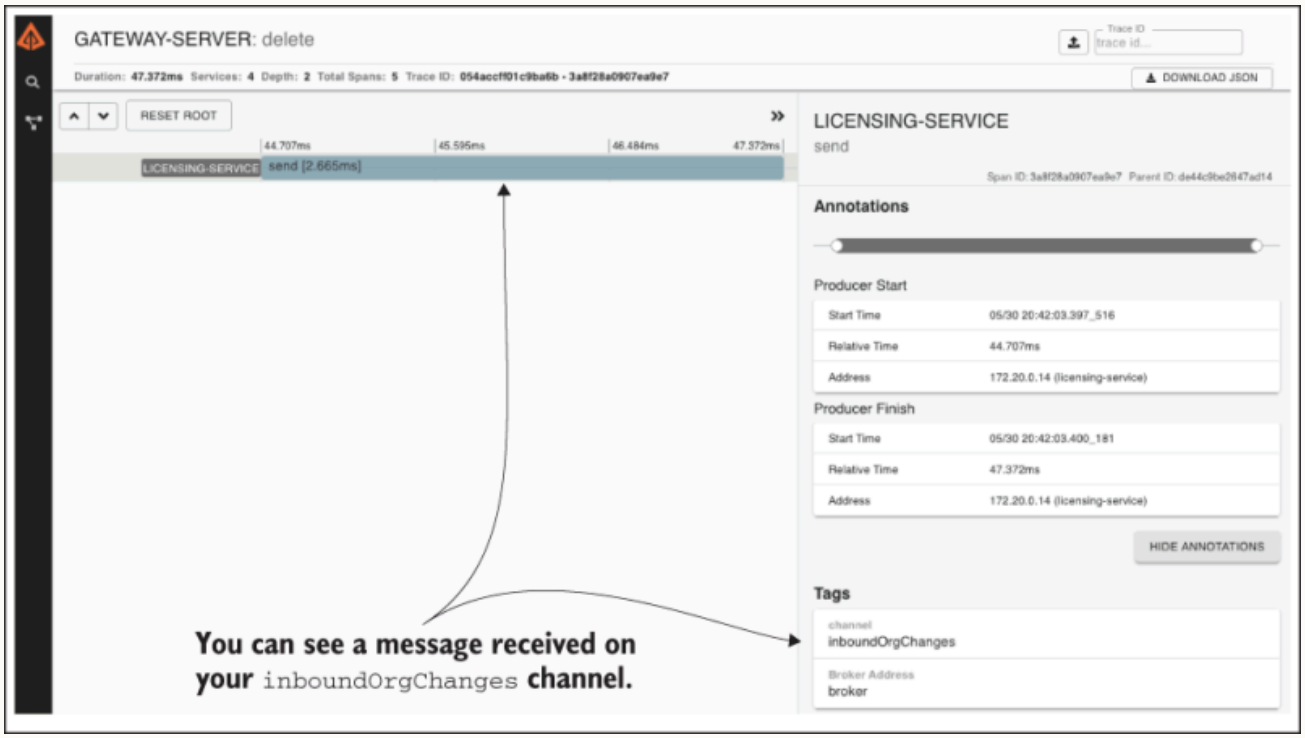


Figure 11.21 Zipkin lets us see the Kafka message being published by the organization service.

Until now, we've used Zipkin to trace our HTTP and messaging calls from within our services. However, what if we want to perform traces out to third-party services that aren't monitored by Zipkin? For example, what if we want to get tracing and timing information for a specific Redis or Postgres SQL call? Fortunately, Spring Cloud Sleuth and Zipkin al-

low us to add custom spans to our transactions so that we can trace the execution time associated with third-party calls.

.3.8 Adding custom spans

Adding a custom span is incredibly easy to do with Zipkin. We'll start by adding a custom span to our licensing service so that we can trace how long it takes to pull data from Redis. Then we'll add a custom span to our organization service to see how long it takes to retrieve data from the organization database. The following listing creates a custom span for the licensing service that's called `readLicensingDataFromRedis`.

Listing 11.8 Adding the Spring Cloud Sleuth trace ID via a response filter

```
package com.optimagrowth.license.service.client;
//Rest of code removed for conciseness

 private Organization checkRedisCache(String organizationId) {
   ScopedSpan newSpan =
       tracer.startScopedSpan
       ("readLicensingDataFromRedis");                    (1)
   try {
    return redisRepository.findById(organizationId).orElse(null);
   } catch (Exception ex){
       logger.error("Error encountered while trying to retrieve
         organization {} check Redis Cache.  Exception {}",
         organizationId, ex);
       return null;
   } finally {
       newSpan.tag("peer.service", "redis");              (2)
       newSpan.annotate("Client received");
       newSpan.finish();                                  (3)
   }

 }
```

(1) Creates a custom span called readLicensingDataFromRedis

(2) Adds tag information to the span and names the service that Zipkin will capture

(3) Closes and finishes the span. If this is not done, we'll get an error message in the log saying that a span was left open.

Next we'll add a custom span, called `getOrgDbCall`, to the organization service to monitor how long it takes to retrieve organization data from the Postgres database. The trace for the organization service database calls can be seen in the `OrganizationService` class, which you'll find in the file /organization-service/src/main/java/com/optimagrowth/organization/service/OrganizationService.java. The `findById()` method call contains the custom trace. The following listing shows the source code for this method.

Listing 11.9 Implementing the `findById()` method

```
package com.optimagrowth.organization.service;
//Some imports omitted for conciseness
import brave.ScopedSpan;
import brave.Tracer;

@Service
public class OrganizationService {
  //Some code omitted for conciseness
  @Autowired
  Tracer tracer;

  public Organization findById(String organizationId) {
    Optional<Organization> opt = null;
    ScopedSpan newSpan = tracer.startScopedSpan("getOrgDBCall");
    try {
      opt = repository.findById(organizationId);
      simpleSourceBean.publishOrganizationChange("GET", organizationId);
      if (!opt.isPresent()) {
        String message = String.format("Unable to find an
           organization with theOrganization id %s",
          organizationId);
        logger.error(message);
        throw new IllegalArgumentException(message);
      }
      logger.debug("Retrieving Organization Info: " +
      opt.get().toString());

    } finally {
        newSpan.tag("peer.service", "postgres");
        newSpan.annotate("Client received");
        newSpan.finish();
      }

    return opt.get();
```

```
    }
    //Rest of class removed for conciseness
}
```

With these two custom spans in place, restart the services. Then select the following GET endpoint:

```
http://localhost:8072/license/v1/organization/4d10ec24-141a-4980-be34-
➥ 2ddb5e0458c9/license/4af05c3b-a0f3-411d-b5ff-892c62710e16
```

If we look at the transaction in Zipkin, we should see the addition of two new spans when we call the licensing service endpoint to retrieve licensing information. Figure 11.22 shows the custom spans.

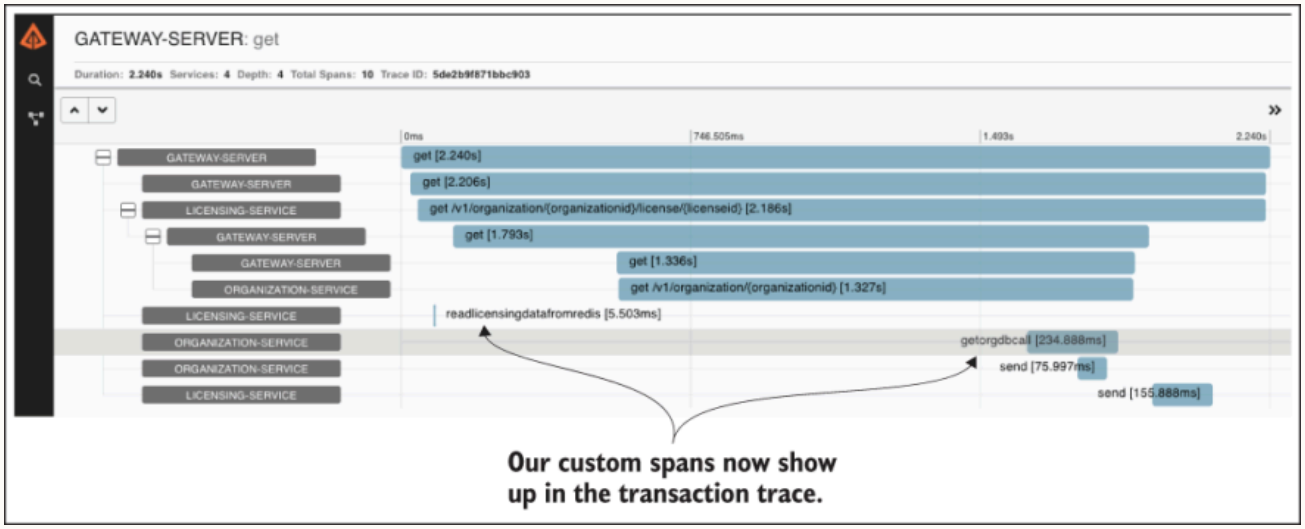


Figure 11.22 The custom spans for our services show up in the transaction trace.

Figure 11.22 also shows additional tracing and timing information related to our Redis and database lookups. Drilling down, we can see that the read call to Redis took 5.503 ms. Because the call didn't find an item in the Redis cache, the SQL call to the Postgres database took 234.888 ms.

Now that we know how to set up a distributed tracing, API gateway, discovery service, and custom spans, let's continue with the next chapter. In the next chapter, we will explain how to deploy everything that we've built throughout the book.

mmary

- Spring Cloud Sleuth allows us to seamlessly add tracing information (correlation IDs) to our microservice calls.

- Correlation IDs can be used to link log entries across multiple services. This lets us observe the behavior of a transaction across all the services involved in a single transaction.
- While correlation IDs are powerful, we need to partner this concept with a log aggregation platform that will allow us to ingest logs from multiple sources and then search and query their contents.
- We can integrate Docker containers with a log aggregation platform to capture all the application logging data.
- We integrate our Docker containers with the ELK (Elasticsearch, Logstash, Kibana) Stack. This lets us transform, store, visualize, and query the logging data from our services.
- While a unified logging platform is essential, the ability to visually trace a transaction through its microservices is also a valuable tool.
- Zipkin allows us to see the dependencies that exist between services and the flow of our transactions, and understand the performance characteristics of each microservice involved in a user's transaction.
- Spring Cloud Sleuth integrates with Zipkin easily. Zipkin automatically captures trace data from an HTTP call, which becomes an inbound/outbound message channel used within a Spring Cloud Sleuth-enabled service.
- Spring Cloud Sleuth maps each service call to the concept of a span. Zipkin then allows us to see the performance of a span.
- Spring Cloud Sleuth and Zipkin also let us define our own custom spans so that we can understand the performance of non-Spring-based resources (a database server such as Postgres or Redis).

Previous chapter
< 10 Event-driven architecture with Spring Cloud Stream

Next chapter
12 Deploying your microservices >